RoboRace

Guia simplificado de desenvolvimento em Python

Objetivo: construir primeiro uma versão funcional no terminal e, só depois, criar a interface gráfica.

1. Ordem recomendada de desenvolvimento

Etapa	O que fazer	Como testar
1. Configuração	Definir constantes, robôs e estado inicial da partida.	Todos começam na posição 0; créditos e acertos começam em 0.
2. Sorteios e movimento	Sortear evento e movimento básico; calcular deslocamento e nova posição.	Testar todos os eventos, incluindo Pane, Óleo e Buraco.
3. Rodada	Processar os três robôs e guardar posição anterior, evento e posição nova.	Confirmar que todos são processados antes de verificar a chegada.
4. Chegada	Identificar todos que alcançaram ou ultrapassaram 30.	Testar vitória, derrota e empate.
5. Desafios	Criar banco de perguntas, sorteio sem repetição e correção da resposta.	Acerto dá 1 crédito; erro não muda créditos nem posição; uma resposta por rodada.
6. Benefícios	Implementar compra e aplicação de Turbo e Escudo.	Testar saldo insuficiente, benefício já equipado e consumo na rodada seguinte.
7. Relatório	Registrar eventos, movimentos, desafio, benefício e saldo.	Conferir saldo inicial + ganhos − gastos = saldo final.
8. Menu terminal	Conectar as funções em um ciclo completo de partida.	Jogar até terminar e poder iniciar outra partida.
9. Estatísticas/exportação	Guardar resultados de partidas e exportar TXT/CSV.	Executar 100 e 1.000 corridas automáticas sem benefícios.
10. Interface gráfica	Reaproveitar a lógica e trocar a apresentação textual por janela e botões.	Não reescrever as regras dentro dos botões da interface.

2. Valores e variáveis principais

Os valores abaixo vêm das regras definidas no relatório RoboRace. Use-os como referência ao criar as constantes e o estado da partida.

Nome sugerido	Tipo Python	Valor inicial / regra
ROBOS	lista de strings	["Azul", "Vermelho", "Verde"]
META	int	30
MOV_MIN / MOV_MAX	int	1 / 3
posicoes	dict[str, int]	Cada robô começa em 0
rodada	int	0
finalizada	bool	False
vencedores	list[str]	Lista vazia
creditos / acertos	int	0 / 0
beneficio	str ou None	None, "Turbo" ou "Escudo"
respondida	bool	False no início de cada rodada
resultado_desafio	str	"não respondido", "acerto" ou "erro"
saldo_inicial / ganhos / gastos	int	Registrar por rodada; ganhos e gastos começam em 0

Nome sugerido	Tipo Python	Valor inicial / regra
relatorios	list	Lista vazia
fila_perguntas	list[int]	Índices embaralhados; renovar ao esgotar
semente	int ou None	Opcional, para repetir sorteios em testes

3. Sorteios: probabilidades e efeitos

Cada robô recebe exatamente um evento por rodada. As probabilidades somam 100%. A forma prática de implementar é usar pesos no sorteio, sem precisar gerar números decimais.

Evento	Chance	Peso sugerido	Efeito
Normal	50%	50	Avança o movimento básico (1 a 3).
Pane	10%	10	Não se move e não sorteia movimento básico.
Turbo (evento)	15%	15	Movimento básico + 3.
Super Turbo	5%	5	Movimento básico + 6.
Óleo	10%	10	Movimento básico − 2, mínimo de 0.
Buraco	10%	10	Movimento básico − 3; posição final nunca fica abaixo de 0.

Movimento básico

Sorteie um inteiro entre 1 e 3 somente se o evento aplicado não for Pane. Para Óleo e Buraco, o deslocamento pode ser negativo, mas a posição final é limitada a zero.

Exemplo de cálculo

Posição anterior = 2; evento Buraco; movimento básico = 1. Deslocamento = 1 − 3 = −2. Nova posição = máximo entre 0 e (2 − 2), portanto 0.

4. Créditos e benefícios

Item	Custo/ganho	Regra importante
Resposta correta	+1 crédito e +1 acerto	No máximo uma resposta por rodada; resposta é opcional.
Resposta errada	0	Não tira créditos e não muda a posição.
Turbo comprado	Custa 2 créditos	Soma +3 à posição depois do efeito do evento, inclusive em Pane.
Escudo comprado	Custa 1 crédito	Transforma Pane, Óleo ou Buraco em Normal para o robô do jogador.

Compra e consumo: descontar os créditos na hora da compra. Só pode haver um benefício equipado. O benefício é aplicado na próxima rodada e consumido nela. O Escudo é consumido mesmo se não bloquear obstáculo.

Ordem por robô: sortear evento → aplicar Escudo se for o jogador → sortear movimento (exceto Pane) → calcular efeito e limitar posição a zero → aplicar Turbo comprado (+3) se for o jogador.

5. Funções sugeridas

Função	Responsabilidade	Prioridade
iniciar_partida()	Criar/resetar o estado da partida.	1
sortear_evento()	Sortear evento usando os pesos definidos.	2
sortear_movimento()	Sortear inteiro entre 1 e 3.	3
calcular_movimento()	Aplicar o evento e calcular a nova posição.	4
jogar_rodada()	Processar os três robôs e guardar os resultados individuais.	5
verificar_chegada()	Encontrar todos com posição >= 30 após a rodada inteira.	6
preparar_rodada()	Salvar saldo inicial, zerar ganhos/gastos e selecionar pergunta.	7
responder_desafio()	Validar resposta, crédito, acerto e explicação.	8
comprar_beneficio()	Validar saldo e benefício equipado; descontar custo.	9
gerar_relatorio()	Montar e guardar o relatório e conferir saldo.	10
resultado_jogador()	Retornar vitória, derrota ou empate.	11
main()	Coordenar menu, ações e repetição de partidas.	12

6. Estrutura de arquivos sugerida

Arquivo	O que fica nele
main.py	Menu do terminal e coordenação do fluxo do jogo.
regras.py	Constantes, probabilidades, custos e definições de eventos.
corrida.py	Estado, sorteios, movimentos, benefícios e chegada.
desafios.py	Banco de perguntas, sorteio, respostas e créditos.
relatorios.py	Relatório por rodada, histórico e exportação TXT/CSV.
testes.py	Testes automáticos das funções e regras.

7. Checklist antes da interface gráfica

☐ A corrida completa funciona no terminal.

☐ Eventos, posições e benefícios foram testados.

☐ Desafios, créditos e compras respeitam as regras.

☐ Empate é verificado depois de processar todos os robôs.

☐ O relatório fecha a conta: saldo inicial + ganhos − gastos = saldo final.

☐ Histórico, estatísticas e exportação TXT/CSV funcionam.

☐ Foram analisadas 100 e 1.000 corridas automáticas sem benefícios.

Regra de ouro: primeiro façam o motor da corrida funcionar e passar nos testes; depois conectem o menu do terminal; por último, construam a interface gráfica reaproveitando as mesmas funções.